Lab 05: Local Search Algorithms

> **Objective:** Investigate how local search algorithms navigate a fitness landscape without retaining a search path. You will use the **Optimization Algorithm Simulator** from the lecture to observe how Hill Climbing, Simulated Annealing and Local Beam Search behave under different parameter settings, explain the trade-offs you observe, and then implement the generate–evaluate–prune cycle of Local Beam Search in Python. Total: **5.0 marks**.

Student Name	HRM A A BANDARA	**Student ID**	IT24100315

Introduction

Local search algorithms do not build or store a path to the goal. They hold one current state (or a small set of states), look only at that state's immediate neighbours, and move according to a selection rule. Because the path is discarded, memory use is constant, which makes these methods suitable for very large or continuous state spaces where systematic search is infeasible. The cost of that efficiency is completeness: the algorithm can settle on a *local* optimum and stop.

> **Hill Climbing (Greedy):** always move to the best-scoring neighbour; halt when no neighbour improves on the current state.
> **Simulated Annealing:** accept an improving move always, and accept a worsening move with probability `P = e^(ΔE / T)`, where the temperature `T` falls over time according to a cooling schedule.
> **Local Beam Search:** keep *k* states, generate all their successors into one shared pool, and retain the best *k* of that pool for the next iteration.

Lab Activity

This is an individual activity. Complete each section on your own and type your answers directly into the boxes below.

Use the Optimization Algorithm Simulator provided in the lecture for Part A. Answer the MCQs in Part B directly in this sheet. For Part C, copy the skeleton code into your local IDE (PyCharm, VS Code, Jupyter or Colab), complete and test it, then paste the working code back into the box provided. Use **Print / Save as PDF** at the top of the page to export your completed sheet for submission.

Section 1: Part A — Simulation Analysis & Parameter Tweaks (2.0 Marks)

1. The Hill Climbing trap and step size (0.5 marks)

Set the simulator to **Hill Climbing (Greedy)** with **Starting X = 100**.
Tweak 1: run with **Step Size = 2** and note where the search becomes trapped.
Tweak 2: change **Step Size = 20** and run again. Does it step over the local maximum?

Run	Final X	Peak reached (local / global)
Step Size = 2	146	Local Maximum
Step Size = 20	140	Local Maximum

Explain the trade-off: how does a larger step size help the search escape a local optimum, and what is the dangerous drawback of setting the step size too large as the search nears the global maximum?

A larger step size can help Hill Climbing escape a narrow local optimum because it evaluates states farther away and may jump into a better region of the fitness landscape. However, if the step size is too large, the search becomes too coarse and can jump over the true global maximum or fail to evaluate states close to its peak. In this simulation, Step Size 20 stopped at X = 140 with fitness 265.29, while Step Size 2 reached X = 146 with the slightly better fitness 266.06. Therefore, the larger step did not escape the local optimum in this run and actually produced a less precise result.

2. Initial condition sensitivity **(0.5 marks)**

Stay on **Hill Climbing** and set **Step Size = 5**.
Tweak 1: start at **X = 50**, run, and record the final peak.
Tweak 2: start at **X = 850**, run, and record the final peak.

Starting X	Final X	Final objective value
50	50	248.38
850	765	281.38

Observe & explain: compare the two final states. Why is greedy local search so sensitive to its initial configuration, and how does Random-Restart Hill Climbing mitigate this specific vulnerability?

Greedy Hill Climbing is very sensitive to its initial state because it only considers neighbouring states and always moves toward an immediate improvement. Starting at X = 50 caused the search to stop immediately at a local optimum with fitness 248.38, while starting at X = 850 led it toward a much better peak at X = 765 with fitness 281.38. Random-Restart Hill Climbing reduces this vulnerability by running Hill Climbing multiple times from different randomly selected starting positions and keeping the best result found.

3. Simulated Annealing: the cooling schedule **(0.5 marks)**

Switch to **Simulated Annealing** with **Starting X = 100** and **Initial Temp = 100**.
Tweak 1: Cooling Rate = 0.50 (fast cooling). Run the simulation.
Tweak 2: Cooling Rate = 0.99 (slow cooling). Run the simulation.

Cooling rate	Final X / value	Behaviour observed
0.50 (fast)	best fitness = 254.85	cooled very quickly; halted after only 8 steps; little exploration.
0.99 (slow)	best fitness = 247.83	explored for 528 steps; accepted many worse moves early; gradually became more selective

Explain the impact: how does the cooling rate control the balance between exploration (random walk) and exploitation (greedy climbing)? Referring to `P = e^(ΔE / T)`, state mathematically what happens to the acceptance probability for a worsening move when the temperature falls too rapidly.

The cooling rate controls how long Simulated Annealing continues exploring. With a low cooling rate such as 0.50, the temperature drops very rapidly, so the algorithm quickly stops accepting worse states and behaves more like greedy Hill Climbing. With a high cooling rate such as 0.99, the temperature decreases slowly, so the algorithm keeps exploring for much longer and can accept worse moves early in the search. In this run, the 0.50 setting halted after only 8 steps, while the 0.99 setting continued for 528 steps. For a worsening move, (Delta E < 0), so (P = e^{`ΔE / T`}). If (T) falls too quickly, (`ΔE / T`) becomes a large negative number and (P ->0), meaning worse moves are almost never accepted.

4. Local Beam Search: the loss of diversity **(0.5 marks)**

Set the simulator to **Local Beam Search** with **Beam Width (k) = 3** and run from a random start. You will most likely see all three agents converge onto the same peak within a few iterations, leaving the rest of the landscape unexplored.

Observe & explain: using the algorithm's successor-selection logic, explain mathematically why this loss of diversity occurs. In your answer, state how the successors of all *k* states are pooled and how the top *k* are chosen.

In Local Beam Search, each of the (k) current states generates successors, and all successors are placed into one shared pool. If each state generates (m) successors, the pool contains up to (k times m) candidates. The algorithm then ranks all candidates by fitness and selects only the top (k) states for the next iteration. Because the best successors often come from the same promising region of the landscape, several of the selected states can become very similar or converge on the same peak. In this run, the top states quickly became concentrated around fitness values 175, 174 and 169, showing how diversity is lost when selection always keeps the absolute best (k) successors.

Section 2: Part B — Theoretical Foundations (MCQs) (1.5 Marks)

Select one option per question and type the letter in the answer box. 0.5 marks each.

5. Which statement best distinguishes Local Search from standard pathfinding (such as A*)?

A) Local search uses heuristics, while standard pathfinding does not.
B) Local search aims to find the optimal sequence of actions, while pathfinding only cares about the final state.
C) Standard pathfinding retains paths in memory to backtrack, whereas local search operates using only the current node (and its neighbours), focusing on the state rather than the path.
D) Local search algorithms are guaranteed to be complete and optimal in continuous spaces.

C

6. In Simulated Annealing the probability of accepting a worse state is `P = e^(ΔE / T)`. As the temperature *T* approaches 0, what happens to the behaviour of the algorithm?

A) It behaves exactly like a purely random walk.
B) It behaves exactly like greedy Hill Climbing, accepting only better states.
C) It accepts all states regardless of their fitness evaluation.
D) The algorithm terminates automatically because ΔE becomes undefined.

B

7. How does Stochastic Beam Search solve the loss-of-diversity problem found in standard Local Beam Search?

A) By choosing the next *k* states randomly, with probability proportional to their fitness, rather than strictly choosing the absolute best *k*.
B) By increasing the beam width *k* dynamically whenever a local minimum is detected.
C) By forcing agents to move in opposite directions (repulsion) when they get too close to each other.
D) By completely resetting the temperature whenever all *k* states converge.

A

Section 3: Part C — Guided Implementation: Local Beam Search (1.5 Marks)

Development instructions:

1. Copy the skeleton below into your local Python IDE (Jupyter, VS Code or Colab).
2. Complete the logic for **generating**, **evaluating** and **pruning** the neighbours.
3. Run the script. The `__main__` block contains a test case (a mathematical parabola); a correct implementation prints a success message.
4. Once it passes, paste your completed code into the box below.

Skeleton code

```
import random

def local_beam_search(initial_states, k, max_iters, objective_fn):
    """
    initial_states: list of k randomly generated starting positions
    k: the beam width (number of states to keep)
    objective_fn: the function we are trying to maximize
    """
    current_states = initial_states

    for iteration in range(max_iters):
        all_successors = []   # Will hold tuples of (state, fitness)

        # --- STEP 1, 2 & 3: Generate, Evaluate, and Pool Neighbors ---
        for state in current_states:
            # Generate 5 random neighbors for this specific state
            for _ in range(5):
                # HINT: neighbor = state + random.uniform(-10, 10)
                # [WRITE CODE HERE]

                # HINT: calc fitness -> fit = objective_fn(neighbor)
                # [WRITE CODE HERE]

                # HINT: append to pool -> all_successors.append((neighbor, fit))
                # [WRITE CODE HERE]
                pass

        # --- STEP 4: Prune the Beam (The Cut) ---
        # HINT 1: Sort the pool descending by fitness.
        #   all_successors.sort(key=lambda x: x[1], reverse=True)
        # [WRITE CODE HERE]

        # HINT 2: Slice the top 'k' items and extract just the state (index 0).
        #   current_states = [item[0] for item in all_successors[:k]]
        # [WRITE CODE HERE]

    return max(current_states, key=objective_fn)


# ======================================================================
# LOCAL TEST RUNNER - DO NOT MODIFY BELOW THIS LINE
# ======================================================================
if __name__ == "__main__":
    # Fix the random seed for reproducible testing
    random.seed(42)
```

Paste your completed, tested code here

```
import random

def local_beam_search(initial_states, k, max_iters, objective_fn):
    """
    initial_states: list of k randomly generated starting positions
    k: the beam width (number of states to keep)
```

```
    objective_fn: the function we are trying to maximize
    """
    current_states = initial_states

    for iteration in range(max_iters):
        all_successors = []   # Will hold tuples of (state, fitness)

        # --- STEP 1, 2 & 3: Generate, Evaluate, and Pool Neighbors ---
        for state in current_states:
            # Generate 5 random neighbors for this specific state
            for _ in range(5):
                # HINT: neighbor = state + random.uniform(-10, 10)
                # [WRITE CODE HERE]
                neighbor = state + random.uniform(-10, 10)

                # HINT: calc fitness -> fit = objective_fn(neighbor)
                # [WRITE CODE HERE]
                fit = objective_fn(neighbor)

                # HINT: append to pool -> all_successors.append((neighbor, fit))
                # [WRITE CODE HERE]
                all_successors.append((neighbor, fit))

        # --- STEP 4: Prune the Beam (The Cut) ---
        # HINT 1: Sort the pool descending by fitness.
        #   all_successors.sort(key=lambda x: x[1], reverse=True)
        # [WRITE CODE HERE]
        all_successors.sort(key=lambda x: x[1], reverse=True)

        # HINT 2: Slice the top 'k' items and extract just the state (index 0).
        #   current_states = [item[0] for item in all_successors[:k]]
        # [WRITE CODE HERE]
        current_states = [item[0] for item in all_successors[:k]]

    return max(current_states, key=objective_fn)


# ================================================================
# LOCAL TEST RUNNER - DO NOT MODIFY BELOW THIS LINE
# ================================================================
if __name__ == "__main__":
    # Fix the random seed for reproducible testing
    random.seed(42)
```

Test output

Paste the exact output printed by the test runner, including the success message and the best state returned.

```
No console output was produced. The provided test runner only initializes random.seed(42) and does not include the referenced parabola test or success-message code.
```

Discussion

Local Beam Search with $k = 3$ is not the same as running Hill Climbing three times independently. Explain the difference and say which one preserves more diversity, and why.

Local Beam Search with (k = 3) keeps three states at the same time, but the states are not independent. In each iteration, all successors generated from the three current states are combined into one shared pool, and only the best three overall are selected for the next iteration. Because of this shared selection, the beam states can quickly converge to the same promising region and lose diversity. In contrast, running Hill Climbing three times independently keeps each search

Your objective function is expensive to evaluate and you are given a fixed budget of evaluations. Which algorithm from this lab would you choose, and how would you set its parameters? Justify your choice using your Part A observations.

If the objective function is expensive to evaluate and the number of evaluations is limited, I would choose Simulated Annealing with a moderate initial temperature and a reasonably slow cooling rate. From Part A, fast cooling at 0.50 stopped after only 8 steps, while slow cooling at 0.99 continued for 528 steps. This shows that the cooling rate strongly affects how much of the evaluation budget is spent

separate, so each run may climb toward a different local optimum. Therefore, three independent Hill Climbing runs generally preserve more diversity than standard Local Beam Search.

on exploration. I would therefore avoid an extremely slow cooling rate because it would consume too many expensive evaluations. I would use a moderate cooling rate so the algorithm can still accept some worse moves early to escape local optima, but gradually become more selective and spend the remaining evaluations exploiting promising regions.